

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
df_features = pd.read_parquet("/content/drive/MyDrive/data/archive/tts_features.parquet")
df_features.head()
```

	mfcc_1_mean	mfcc_2_mean	mfcc_3_mean	mfcc_4_mean	mfcc_5_mean	mfcc_6_mean	mfcc_7_mean	mfcc_8_mean	mfcc_9_mean
0	-231.092453	114.497093	7.188296	36.901474	15.573195	-2.370653	-16.082829	-22.349562	-6.497463
1	-378.915619	179.476929	7.342104	22.781954	-10.691727	-15.086653	-10.566176	-16.604702	-12.440489
2	-398.897552	123.861023	24.483349	56.993763	30.592896	-6.037459	-16.340183	7.622237	-5.345806
3	-389.487488	112.788864	9.360864	38.784508	11.073611	3.233735	-8.266920	-0.416876	-7.667834
4	-270.491730	125.467178	-7.004885	14.316159	16.208467	-23.108959	-11.284864	-13.697815	-19.507792

5 rows × 89 columns

```
X = df_features.drop(columns=["label"])
y = df_features["label"]

print(type(X))
print(type(y))
print(X.shape)
print(y.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
<class 'pandas.core.series.Series'>
(34850, 88)
(34850,)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size = 0.2, random_state= 42, stratify =y)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(27880, 88)
(6970, 88)
(27880,)
(6970,)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix

rf = RandomForestClassifier(n_estimators=200,random_state=42,n_jobs=-1)
rf.fit(X_train,y_train)
y_pred = rf.predict(X_test)
acc = accuracy_score(y_test,y_pred)
print(f"Accuracy: {acc:4f}")
print(classification_report(y_test,y_pred))
cm = confusion_matrix(y_test,y_pred)
print(cm.shape)
print(cm)

importance = rf.feature_importances_
importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importance
})
importance_df.sort_values(by="Importance",ascending=False,inplace=True)
importance_df.head(10)
```


Accuracy: 0.870732

	precision	recall	f1-score	support
cv4_fastspeech2	0.98	0.99	0.99	278
ek1_tacotron2	1.00	1.00	1.00	279
ljspeech_deepvoice3	1.00	1.00	1.00	279
ljspeech_fast_pitch	0.77	0.73	0.75	279
ljspeech_fastspeech2	0.96	0.95	0.96	279
ljspeech_glow-tts	0.82	0.87	0.85	279
ljspeech_tacotron2-DCA	0.85	0.84	0.84	279
ljspeech_tacotron2-DDC	0.80	0.76	0.78	279
ljspeech_tacotron2-DDC_ph	0.88	0.88	0.88	278
ljspeech_vits	0.84	0.86	0.85	279
sam_tacotron-DDC	1.00	1.00	1.00	279
silero_lj_16khz	0.91	0.96	0.93	278
silero_lj_16khz_enhanced	0.99	0.97	0.98	279
tf_ljspeech_FASTSPEECH2_MB-MELGAN	0.81	0.83	0.82	279
tf_ljspeech_FASTSPEECH2_MELGAN	0.70	0.69	0.70	279
tf_ljspeech_FASTSPEECH2_MELGAN-STFT	0.73	0.68	0.71	279
tf_ljspeech_FASTSPEECH2_MB-MELGAN	0.78	0.85	0.81	278
tf_ljspeech_FASTSPEECH2_MELGAN	0.75	0.68	0.72	279
tf_ljspeech_FASTSPEECH2_MELGAN-STFT	0.73	0.72	0.73	279

Next steps: [Generate code with importances and](#)[New interactive sheet](#)

!pip install xgboost

vctk_deepvoices_speakersrandom 0.99 0.99 0.99 279

```

from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

```

```

le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

```

```

xgb = XGBClassifier(objective = "multi:softmax", num_class = 25, n_estimators = 200, max_depth = 8, learning_rate=0.1,
xgb.fit(X_train, y_train_enc)
y_pred = xgb.predict(X_test)
acc = accuracy_score(y_test_enc, y_pred)
print(f"Accuracy: {acc:4f}")

```

```

y_pred_labels = le.inverse_transform(y_pred)
print(classification_report(y_test, y_pred_labels))
cm = confusion_matrix(y_test, y_pred_labels)
print(cm.shape)
print(cm)

```

```

[[ 0  0  0  0  27  0  4 10 212 10 14  0  0  0  0  0  1  0  0
Accuracy: 0.870732
[[ 1  0  0  4  0  1  3 12 246
precision 0 recall 0 f1score 0 support
[[ 0  1  1  0  0  0  0  0
cv4_fastspeech2 8 240 0.99 0 0.99 0 0.99 0 278
ek1_tacotron2 1.00 1.00 1.00 279
ljspeech_deepvoice3 1.00 1.00 1.00 279
ljspeech_fast_pitch 0.77 0.73 0.75 279
ljspeech_fastspeech2 0.96 0.95 0.96 279
ljspeech_glow-tts 0.82 0.87 0.85 279
ljspeech_tacotron2-DCA 0.85 0.84 0.84 279
ljspeech_tacotron2-DDC 0.80 0.76 0.78 279
ljspeech_tacotron2-DDC_ph 0.88 0.88 0.88 278
ljspeech_vits 0.84 0.86 0.85 279
sam_tacotron-DDC 1.00 1.00 1.00 279
silero_lj_16khz 0.91 0.96 0.93 278
silero_lj_16khz_enhanced 0.99 0.97 0.98 279
tf_ljspeech_FASTSPEECH2_MB-MELGAN 0.81 0.83 0.82 279
tf_ljspeech_FASTSPEECH2_MELGAN 0.70 0.69 0.70 279
tf_ljspeech_FASTSPEECH2_MELGAN-STFT 0.73 0.68 0.71 279
tf_ljspeech_FASTSPEECH2_MB-MELGAN 0.78 0.85 0.81 278
tf_ljspeech_FASTSPEECH2_MELGAN 0.75 0.68 0.72 279
tf_ljspeech_FASTSPEECH2_MELGAN-STFT 0.73 0.72 0.73 279
vctk_deepvoices_speakersrandom 0.99 0.99 0.99 279
vctk_fast_pitch_speakersrandom 1.00 1.00 1.00 279
vctk_glow-tts_speakersrandom 0.99 0.97 0.98 278
vctk_vits_speakersrandom 0.97 0.99 0.98 279
accuracy 0.870732
macro avg 0.91 0.91 0.91 279
weighted avg 0.91 0.91 0.91 279
(25, 25)
[[278  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0

```